

FreeFlume — Multi-Audio, Downloads & Shorts/UX Changes

Work log covering the multi-audio (dub) feature through the Shorts and player-UX changes. Written 2026-07-01.

Overview

This round of work added four capability areas to FreeFlume:

1. **Multi-audio playback** — pick a YouTube audio-language track ("dub") in the player, including AI auto-dubs.
2. **Multi-audio downloads** — download a chosen dub, or embed every language into one file; with clearer multi-stream progress.
3. **YouTube Shorts** — a channel Shorts tab, playable Shorts links, and Shorts kept out of the subscription feed.
4. **Player UX** — keep a maximized window maximized after fullscreen, and close the mini-player when a finished video is backed out of.

Two commits landed for the audio work (`af2129e` , `c6a0139`); the Shorts/UX batch is built and awaiting a commit.

1. Multi-audio playback (dubs)

The core discovery

YouTube exposes alternate-language audio ("dubs"), but **yt-dlp's default player client hides them** — it returns only the original track. The alternate languages are surfaced by a specific client:

```
--extractor-args youtube:player_client=web_embedded
```

This was verified live: on one test video the default client returned only Japanese, while `web_embedded` returned both the Japanese original and the English dub, each with a working stream URL — **no login, no PO token, no auth**. Other clients (`web` , `tv` , `mweb`) are PO-token-gated and returned nothing.

This mattered because the earlier attempt at this feature failed for exactly one reason: the wrong client. It also sidesteps the NewPipe-style approach of talking to InnerTube directly, which would mean owning the PO-token "treadmill" — a poor fit for a low-ceremony, no-auth app.

Original vs. dubbed vs. auto-dubbed

yt-dlp's `format_note` does **not** distinguish a manual dub from a machine (auto) dub — both just show the language name. The real signal lives in the stream URL's `xtags` parameter:

acount value	Meaning
original	the video's original audio
dubbed	a manual / licensed dub
dubbed-auto	YouTube's machine-generated dub

`language_preference == 10` also flags the original. FreeFlume parses `acount` out of the URL (via `UrlQuery`) to label tracks correctly.

How playback switches languages

Playback routes through mpv's built-in `ytdl_hook`, so switching a dub is done by constraining the format selector and reloading:

- **Discovery** — `Extractor::fetchAudioTracks()` runs a `web_embedded` probe and parses the distinct audio-format languages (only the *list* is needed here, so `nsig/stream` URLs don't matter). Original(s) sort to the top, the rest alphabetical.
- **UI** — an `AUD` button in the transport bar, hidden unless a video has more than one language. It shows the active language as a 3-letter tag (`ENG` / `JPN` / `FRA`). The menu labels tracks "X (original)", "X (auto-dubbed)", or plain "X" for a manual dub.
- **Switching** — `MpvWidget::setAudioMultiClient(true)` adds `web_embedded` to mpv's client list, the `ytdl-format` gains a `[language=CODE]` filter, and the existing seamless `reload()` (resume at current position) applies it. Default playback stays on the fast `default,android` client with the original audio — no regression; `web_embedded` is only engaged when a dub is picked.

Files: `extractor.{h,cpp}`, `mpvwidget.{h,cpp}`, `playerpage.{h,cpp}`.

2. Multi-audio downloads

Before this, downloads always fetched the original audio. Two tiers were added.

Tier 1 — download the selected language

A download now inherits the audio language chosen in the player:

- The video/audio selectors gain a `[language=CODE]` filter and the `web_embedded` client, threaded through `downloadmenu::addSubmenu(..., audioLang)` and `DownloadManager::enqueue(..., audioLang)`.
- Fallback order is dub → high-quality original → progressive single file, so a missing dub degrades to good original audio rather than a low-res copy.
- Dubbed files are tagged so they don't collide: `Title.<lang>.ext`.

- Search-result and channel-list download menus are unchanged (default = original).

Tier 2 — embed all languages

Settings → Downloads gained a pair of options:

- "Embed all audio languages in video downloads"
- "Include auto-dubbed audio" (nested, indented, **off by default**)

When on, a video download first runs a `web_embedded` probe, keeps the original plus manual dubs (auto-dubs only when opted in), and muxes them into a single **MKV** via `--audio-multistreams`.

Honesty note surfaced in the tooltip: most videos today are auto-dub-only, so with the auto option **off**, "embed all languages" often yields just the original. The tooltip warns about this rather than silently producing a one-language file.

The `--embed-metadata` fix

The first embed-all attempt tagged **every** audio track as `eng` in the muxed MKV. yt-dlp does not propagate per-stream language on merge unless told to. Fix: add `--embed-metadata`, after which each track carries its real ISO-639-2 code (`en-US` → `eng`, `fr-FR` → `fra`, `ja` → `jpn`) — verified with `ffprobe`.

Progress UX

Multi-stream downloads used to look like the video was downloading repeatedly (each stream has its own 0–100% pass). Fixed:

- **Aggregate progress** — the per-stream percent is combined across all parts, so the bar climbs `0→100` once. Parsed from yt-dlp's `Downloading N format(s): a+b+c` line and each `Destination: .`
- **Clearer status** — the download list shows "track *i/N*" and a distinct "Merging tracks..." phase.
- **Sidebar indicator** — a click-through active-download widget (caption + progress bar) below the nav list, opening the Downloads page on click.

Files: `downloadmanager.{h,cpp}`, `downloadmenu.h`, `downloadspage.cpp`, `mainwindow.{h,cpp}`, plus a shared `parseAudioTracks()` factored into `extractor.{h,cpp}`.

3. YouTube Shorts

The key constraint

yt-dlp's search returns no Shorts at all — not as `/shorts/` URLs, not as short `/watch` videos. The Shorts shelf is dropped from both `ytsearchN:` and the results-page path. Verified empirically (zero Shorts in either).

So "show Shorts in search results" is not achievable through the extractor without a custom InnerTube Shorts-shelf parser. Shorts *do* appear cleanly in a channel's **/shorts** tab (proper `/shorts/` URLs) and via direct links.

What was built

- **Channel Shorts tab.** A `ChannelTab` enum `{Videos, Streams, Shorts}` replaced the old `Videos/Streams` boolean across `Extractor::fetchChannel` and the search page. Channel pages now show **Videos / Streams / Shorts** tabs; the Shorts tab loads `channelBaseUrl + /shorts`. (Originally gated behind a setting — the setting was removed once we learned search can't surface Shorts anyway; a click-to-open tab is unobtrusive enough not to need a toggle.)
- **Shorts links play.** `share::videoId()` now recognizes `/shorts/`, `/live/`, and `/embed/` path forms (previously only `watch?v=` and `youtu.be`), so a pasted Shorts link plays directly.

Keeping Shorts out of subscriptions

The subscription feed is built from each channel's **RSS** (`feeds/videos.xml`), which **mixes Shorts into uploads with no marker** — on one test channel, 11 of 15 RSS entries were Shorts. RSS carries no distinguishing field (the `640×390 media:content` is a fixed placeholder).

Classification uses a cheap redirect probe: `youtube.com/shorts/<id>` returns **200** for a real Short and **303 → /watch** for a normal video. A `HEAD` request with `ManualRedirectPolicy` reads the status directly.

- Results are cached in the DB (`meta_cache.is_short`, a new column), since a video's Shorts-ness never changes — so only **brand-new** items are ever checked, and steady-state refreshes are instant.
- `Database::cachedIsShort()` returns `1 / 0 / -1` (short / not / unknown); `cacheUploadDate` was switched to an `ON CONFLICT upsert` so it can't clobber the flag.
- `SubscriptionFeed::classifyShorts()` checks uncached items, then `emitFiltered()` drops confirmed Shorts. Unknowns and transient errors are **kept**, so a real upload is never hidden by mistake.

Files: `extractor.{h,cpp}`, `searchpage.{h,cpp}`, `sharemenu.h`, `settingspage.{h,cpp}`, `subscriptionfeed.{h,cpp}`, `database.{h,cpp}`.

4. Player UX fixes

- **Maximized survives fullscreen.** Exiting fullscreen always called `showNormal()`, losing a maximized window. Now `MainWindow` records `wasMaximized_ = isMaximized()` on entering fullscreen and calls `showMaximized()` vs `showNormal()` on exit.
- **Back after end closes the mini-player.** When a video finishes with nothing queued next, `PlayerPage` emits `playbackFinished()`. `MainWindow` tracks `playbackEnded_` (cleared when a new video starts), and `collapsePlayer()` goes to *Hidden* instead of *Mini* when set

— so backing out of a finished video closes the player instead of leaving a dead mini-player. Mid-video back still goes to mini as before.

Files: `mainwindow.{h,cpp}` , `playerpage.{h,cpp}` .

Testing checklist

Area	Test
Dub playback	Play a multi-audio video → AUD button shows active language; switching reloads seamlessly.
Auto-dub labels	An auto-dub video lists "X (auto-dubbed)"; the original shows "X (original)".
Download a dub	Select a dub, download → file tagged <code>...<lang>...</code> with that audio.
Embed all	Settings → embed all; a manual-dub video yields an MKV with multiple correctly-tagged audio tracks.
Progress	Multi-stream download shows one rising bar, "track i/N", then "Merging tracks...".
Sidebar indicator	Appears while downloading; click opens Downloads.
Channel Shorts	Channel page shows Videos/Streams/Shorts; Shorts tab loads and plays.
Shorts link	Pasting a <code>youtube.com/shorts/...</code> link plays it.
Subscriptions	"What's New" excludes Shorts (first refresh classifies new items, then cached).
Maximize + fullscreen	Maximize → fullscreen → exit → stays maximized.
Mini-player on end	Let a video finish, click back → player closes (not mini).

Key takeaways

- **web_embedded** is the client that exposes YouTube dubs — without it the default client hides them. The earlier attempt failed purely on client choice.
- **acon** in the stream URL's `xtags` is the only reliable original / dubbed / auto-dubbed signal; `format_note` isn't enough.
- **--embed-metadata** is required or a multi-audio merge tags every track `eng` .
- **yt-dlp search omits Shorts**, so Shorts only reach the app via channel tabs and direct links — and the subscription RSS silently mixes them in, needing a per-video redirect probe (cached) to filter out.